System Design: LLD + HLD

Complete interview roadmap for Python Web Development, AI Engineering & Data Science

Scope: Low-Level Design (LLD), High-Level Design (HLD), distributed systems, databases, APIs, AI/ML systems, data platforms, scalability, reliability, security and interview practice.

How to use this PDF: Study fundamentals → LLD → HLD → domain-specific systems → design case studies → mock interviews. For each topic, practice explaining what it is, why it is needed, trade-offs, failure modes and a Python implementation where relevant.

1. System Design Fundamentals

- System design vs software architecture vs detailed design
- Functional requirements and non-functional requirements (NFRs)
- Availability, reliability, scalability, durability, consistency, latency and throughput
- Capacity estimation: users, requests/sec, storage, bandwidth, compute
- Back-of-the-envelope calculations
- Vertical vs horizontal scaling
- Stateless vs stateful services
- Single service vs modular monolith vs microservices
- Synchronous vs asynchronous communication
- Push vs pull architectures
- Read-heavy vs write-heavy workloads
- Hotspots, bottlenecks and single points of failure
- CAP theorem, PACELC and practical consistency choices
- Failure domains and blast radius
- Observability-driven design
- Cost-aware architecture

2. LLD Core Concepts

- OOP principles: encapsulation, abstraction, inheritance, polymorphism
- SOLID principles with Python examples
- Composition over inheritance
- Coupling and cohesion
- Interfaces, abstract base classes and protocols
- Dependency injection and inversion of control
- Immutability and value objects
- Separation of concerns
- Layered architecture
- Clean Architecture
- Hexagonal / Ports-and-Adapters architecture
- Domain-driven design basics
- Entities, value objects, aggregates, repositories and services
- DTOs, domain models and persistence models
- Error handling and exception hierarchy
- Validation and business rules
- Configuration management

- Logging and auditability
- Idempotency at object/service level
- Unit-testable design and mocking

3. LLD Design Patterns

- Creational: Factory, Abstract Factory, Builder, Prototype, Singleton
- Structural: Adapter, Decorator, Facade, Proxy, Composite
- Behavioral: Strategy, Observer, Command, State, Template Method, Chain of Responsibility, Mediator
- Repository pattern
- Service layer pattern
- Unit of Work pattern
- Specification pattern
- Dependency Injection pattern
- Repository + Unit of Work with SQLAlchemy
- Plugin architecture
- Event-driven domain design
- Pattern selection: when NOT to use a pattern

4. Python-Specific LLD

- Python classes, dataclasses and enums
- abc.ABC and abstractmethod
- typing, Protocol, Generic, TypeVar and type aliases
- Context managers and contextlib
- Decorators and middleware-style design
- Iterators, generators and lazy evaluation
- AsyncIO, async/await and cancellation
- Threading vs multiprocessing vs asyncio
- Dependency injection in FastAPI
- Pydantic models and validation
- SQLAlchemy ORM design
- Repository and transaction boundaries
- Package/module structure for large Python projects
- Configuration with environment variables and settings objects
- Linting, formatting, type checking and test architecture

5. Common LLD Interview Problems

- Parking lot

- Library management system
- Banking / ATM system
- Vending machine
- Elevator system
- Movie ticket booking
- Restaurant / food ordering
- Hotel booking
- Ride-sharing domain model
- Payment processing
- Notification service
- Rate limiter
- URL shortener internals
- File storage abstraction
- Task scheduler
- Job queue
- Cache abstraction
- Chat / messaging domain
- Shopping cart and order management
- Inventory management

6. HLD Core Building Blocks

- Clients, DNS, CDN, load balancers and API gateways
- Reverse proxy and TLS termination
- Application servers and worker processes
- Service discovery
- REST, GraphQL and gRPC
- WebSockets and Server-Sent Events
- Message queues and event streaming
- Kafka concepts: topics, partitions, consumer groups, offsets and retention
- RabbitMQ concepts: exchanges, queues, routing and acknowledgements
- Databases and caching layers
- Object storage and file processing
- Search engines
- Schedulers and background workers
- Feature flags and configuration services
- Secrets and key management
- Observability stack

- Containerization and orchestration
- Multi-region architecture
- Disaster recovery

7. API & Web System Design

- REST resource modeling
- HTTP methods, status codes, headers and caching
- Pagination: offset vs cursor
- Filtering, sorting and search
- API versioning
- Idempotency keys
- Retries, timeouts and exponential backoff
- Rate limiting and quotas
- Authentication: sessions, JWT and OAuth2/OIDC
- Authorization: RBAC and ABAC
- CORS, CSRF, XSS and common API threats
- Webhooks and webhook retries
- File upload/download architecture
- Long-running jobs and polling
- WebSockets for realtime systems
- API gateway responsibilities
- OpenAPI / Swagger

8. Databases & Data Modeling

- Relational vs NoSQL databases
- PostgreSQL/MySQL fundamentals
- Tables, keys, constraints and normalization
- Indexes: B-tree, hash, composite and partial indexes
- Query planning and EXPLAIN
- Transactions and ACID
- Isolation levels and locking
- Deadlocks
- Optimistic vs pessimistic concurrency
- Replication: primary/replica
- Read replicas and write scaling
- Sharding and partitioning
- Connection pooling

- Database migrations
- Redis: cache, session, counters, locks and streams
- Document databases
- Wide-column databases
- Time-series databases
- Vector databases for AI
- Polyglot persistence

9. Caching & Performance

- Cache-aside, read-through and write-through
- Write-back and write-around caching
- TTL and eviction policies
- LRU/LFU and cache sizing
- Cache invalidation strategies
- Cache stampede / thundering herd
- Hot keys
- Distributed locks
- Local cache vs distributed cache
- CDN caching
- HTTP caching and ETags
- Database query optimization
- Connection pools and worker pools
- Profiling Python applications
- Latency budgets and p95/p99

10. Distributed Systems

- Service boundaries
- Distributed transactions
- Sagas and compensating actions
- Two-phase commit basics
- Eventual consistency
- Strong vs causal consistency
- Leader election
- Consensus basics
- Replication and quorum
- Distributed locks
- Idempotent consumers

- Exactly-once vs at-least-once vs at-most-once semantics
- Message ordering
- Deduplication
- Outbox pattern
- CDC (Change Data Capture)
- Backpressure
- Retry storms
- Circuit breaker, bulkhead and fallback
- Graceful degradation

11. Reliability, Observability & Operations

- SLO, SLA and SLI
- Error budgets
- Health checks and readiness/liveness
- Structured logging
- Metrics and dashboards
- Distributed tracing
- Correlation/request IDs
- Alerting
- OpenTelemetry concepts
- Failure injection and chaos testing
- Blue-green deployment
- Canary deployment
- Rolling deployment
- Autoscaling
- Capacity planning
- Backup and restore
- RPO and RTO
- Disaster recovery and business continuity

12. Security System Design

- Threat modeling
- Authentication vs authorization
- OAuth2 and OIDC
- JWT security
- API keys and service identities
- RBAC/ABAC

- Password hashing and secrets
- Encryption in transit and at rest
- Key rotation
- Input validation
- SQL injection, XSS, CSRF, SSRF
- Rate limiting and abuse prevention
- PII handling and data minimization
- Audit logs
- Tenant isolation
- Secure file uploads
- Dependency and supply-chain security
- Security boundaries in microservices

13. Python Web Development HLD

- Django architecture: URLs, views, models, services and middleware
- FastAPI architecture: routers, dependencies, schemas and services
- ASGI vs WSGI
- Uvicorn/Gunicorn deployment concepts
- Background tasks and Celery
- Redis-backed task queues
- PostgreSQL + SQLAlchemy
- Connection pooling
- WebSocket architecture
- API gateway + FastAPI services
- Monolith-to-microservices migration
- File processing pipeline
- Email/SMS/notification architecture
- Multi-tenant SaaS backend
- High-traffic e-commerce backend
- Payment/order/inventory consistency

14. AI Engineer System Design

- LLM application architecture
- Prompt → retrieval → model → validation → response pipeline
- RAG architecture
- Document ingestion pipeline
- Parsing, chunking and metadata

- Embedding generation and model selection
- Vector database indexing and retrieval
- Dense, sparse and hybrid retrieval
- BM25 + vector search
- Reciprocal Rank Fusion
- Reranking
- Query rewriting
- HyDE
- Multi-query retrieval
- Parent-document and sentence-window retrieval
- Context compression
- Context window management
- Semantic caching
- LLM gateway and model routing
- Fallback models
- Streaming responses
- Tool/function calling
- Agent architecture and state management
- Human-in-the-loop
- Guardrails and PII protection
- Prompt injection and data exfiltration defenses
- Evaluation: offline + online
- RAG evaluation: faithfulness, relevance, recall and groundedness
- Tracing and cost/latency monitoring
- LLM observability
- Fine-tuning vs RAG vs prompt engineering
- Quantization and inference optimization
- GPU serving and batching
- Model versioning and rollback

15. AI/ML Serving HLD

- Training vs inference architecture
- Batch vs online inference
- Feature extraction pipeline
- Model registry
- Model artifact storage
- Model serving API

- Autoscaling inference workers
- GPU vs CPU inference
- Dynamic batching
- Async inference
- Model caching
- Canary model rollout
- A/B testing models
- Shadow deployment
- Drift monitoring
- Data quality monitoring
- Feature store concepts
- Online/offline feature consistency
- ML pipeline orchestration
- Retraining triggers

16. Data Science / Data Engineering System Design

- OLTP vs OLAP
- Data warehouse vs data lake vs lakehouse
- ETL vs ELT
- Batch processing
- Streaming processing
- Data ingestion architecture
- CDC pipelines
- Kafka-based streaming
- Spark fundamentals
- Partitioning and file formats
- Parquet and columnar storage
- Data modeling: star and snowflake schemas
- Fact and dimension tables
- Slowly changing dimensions
- Data quality checks
- Schema evolution
- Data lineage and catalog
- Orchestration with Airflow
- Data pipeline retries and idempotency
- Late-arriving data
- Exactly-once challenges

- BI/dashboard serving layer
- Real-time analytics
- Data governance and PII

17. AI + Data Platform Design

- Enterprise RAG platform
- Central document ingestion service
- Multi-source connectors
- Metadata store + object storage + vector DB
- Hybrid retrieval + reranking
- LLM gateway
- Prompt/version registry
- Evaluation platform
- Feedback loop
- Conversation memory
- Semantic cache
- Audit and access control
- Multi-tenant knowledge bases
- Real-time recommendation system
- Fraud detection pipeline
- Customer churn prediction platform
- Demand forecasting system
- Personalized recommendation engine
- Real-time anomaly detection

18. Cloud & DevOps HLD

- AWS core services: EC2, ECS/EKS, Lambda, S3, RDS, ElastiCache, SQS, SNS, MSK
- VPC, subnets, security groups and load balancers
- IAM and least privilege
- Containers and Docker
- Kubernetes: pods, deployments, services, ingress, config and secrets
- Horizontal Pod Autoscaler
- Terraform / Infrastructure as Code
- CI/CD with GitHub Actions
- Artifact registries
- Environment separation
- Blue-green/canary deployments

- Managed databases and backups
- Cloud cost optimization

19. Architecture Styles to Know

- Monolith
- Modular monolith
- Microservices
- Serverless
- Event-driven architecture
- Layered architecture
- Clean architecture
- Hexagonal architecture
- Pipe-and-filter
- Batch architecture
- Lambda architecture
- Kappa architecture
- Data mesh basics
- Service-oriented architecture

20. HLD Interview Case Studies

- URL shortener
- Twitter/X-like feed
- Instagram-like photo platform
- YouTube/video streaming
- Netflix-like streaming
- WhatsApp/chat system
- Uber/ride sharing
- Food delivery
- Amazon/e-commerce
- Payment gateway
- Ticket booking
- Notification platform
- Search/autocomplete
- News feed
- Rate limiter
- Distributed cache
- File storage like S3

- Log aggregation system
- Metrics/monitoring platform
- Job scheduler
- Web crawler
- Recommendation system
- Fraud detection
- Real-time analytics
- Enterprise RAG
- AI customer-support agent
- LLM gateway
- Model serving platform
- ML feature store
- Data lakehouse

21. Interview Answer Framework

- Step 1: Clarify functional requirements
- Step 2: Clarify non-functional requirements
- Step 3: Estimate scale
- Step 4: Define APIs/interfaces
- Step 5: Design data model
- Step 6: Draw high-level architecture
- Step 7: Deep dive into the hardest component
- Step 8: Discuss caching, queues and databases
- Step 9: Discuss consistency and concurrency
- Step 10: Discuss failure handling
- Step 11: Discuss security
- Step 12: Discuss observability
- Step 13: Discuss scaling and cost
- Step 14: Summarize trade-offs

22. Must-Know Trade-offs

- SQL vs NoSQL
- Redis vs database
- REST vs gRPC vs GraphQL
- Kafka vs RabbitMQ
- Monolith vs microservices
- Sync vs async

- Polling vs WebSocket vs SSE
- Strong vs eventual consistency
- Offset vs cursor pagination
- Read replica vs sharding
- Cache vs database
- RAG vs fine-tuning
- Dense vs sparse vs hybrid retrieval
- Small LLM vs large LLM
- CPU vs GPU inference
- Batch vs real-time processing
- Lambda vs Kappa architecture
- Self-hosted vs managed infrastructure

23. Practice Checklist

- Draw every architecture without looking at notes
- Explain each component in one sentence
- State at least 2 alternatives and why you rejected them
- Calculate approximate QPS/storage/bandwidth
- Identify at least 3 failure scenarios
- Identify bottlenecks and single points of failure
- Define consistency and durability requirements
- Add caching only where it solves a measured problem
- Add queues for decoupling and burst absorption
- Explain data partitioning strategy
- Explain security boundaries
- Explain monitoring and alerting
- For AI systems, explain retrieval, model, evaluation, safety and cost
- For data systems, explain ingestion, storage, processing, serving and governance

24. Suggested Study Order

Phase 1 — Foundations: OOP, SOLID, Python typing, APIs, HTTP, SQL, Redis, networking basics.

Phase 2 — LLD: Design patterns → clean architecture → repositories/services → 10 LLD problems.

Phase 3 — HLD: Scalability → load balancing → caching → queues → databases → distributed systems.

Phase 4 — Web: Django/FastAPI production architecture, auth, async jobs, WebSockets, observability.

Phase 5 — AI: RAG → hybrid retrieval → reranking → agents → guardrails → evaluation → serving.

Phase 6 — Data Science: ETL/ELT → warehouse/lakehouse → Spark → streaming → feature/data pipelines.

Phase 7 — Cloud: Docker → AWS → Kubernetes → Terraform → CI/CD → monitoring.

Phase 8 — Interviews: Daily LLD + HLD case studies; practice requirements, estimates, architecture and trade-offs.

Quick priority list for a Python + AI Engineer

Highest priority: Python OOP/SOLID, design patterns, FastAPI/Django architecture, REST, PostgreSQL, Redis, Kafka, Docker, Kubernetes basics, system scalability, distributed systems, RAG architecture, vector databases, hybrid search, reranking, LLM gateways, evaluation, model serving, data pipelines and AWS.

This is an interview-oriented topic checklist, not a claim that every topic must be mastered at the same depth. LLD focuses on code/object design; HLD focuses on components, data flow, scale, reliability and trade-offs.